What is DP?

- Wikipedia definition: "method for solving complex problems by breaking them down into simpler subproblems"

- This definition will make sense once we see some examples
  - Actually, we'll only see problem solving examples today

Steps for Solving DP Problems

1. Define subproblems
2. Write down the recurrence that relates subproblems
3. Recognize and solve the base cases

- Each step is very important!

Dynamic programming

- Main approach: recursive, holds answers to a sub problem in a table, can be used without recomputing.

- Can be formulated both via recursion and saving results in a table (*memoization*). Typically, we first formulate the recursive solution and then turn it into recursion plus dynamic programming via *memoization* or bottom-up.

Comparison with divide-and-conquer

- In contrast, a dynamic programming algorithm proceeds by solving small problems, remembering the results, then combining them to find the solution to larger problems
- Dynamic programming can be thought of as bottom-up

The principle of optimality, I

- Dynamic programming is a technique for finding an *optimal* solution
- The principle of optimality applies if the optimal solution to a problem can be obtained by combining the optimal solutions to all subproblems.

Steps for Solving DP Problems

- Define subproblems
- Write down the recurrence that relates subproblems
- Recognize and solve the base cases

- Each step is very important.

- Dynamic programming relies on working “from the bottom up” and saving the results of solving simpler problems
  - These solutions to simpler problems are then used to compute the solution to more complex problems
- Dynamic programming solutions can often be quite complex and tricky

- Dynamic programming is used for optimization problems, especially ones that would otherwise take exponential time
  - Only problems that satisfy the principle of optimality are suitable for dynamic programming solutions
- Since exponential time is unacceptable for all but the smallest problems, dynamic programming is sometimes essential.

Dynamic programming

- Dynamic programming relies on working "from the bottom up" and saving the results of solving simpler problems
  - These solutions to simpler problems are then used to compute the solution to more complex problems
- Dynamic programming solutions can often be quite complex and tricky
- Dynamic programming is used for optimization problems, especially ones that would otherwise take exponential time
  - Only problems that satisfy the principle of optimality are suitable for dynamic programming solutions
- Since exponential time is unacceptable for all but the smallest problems, dynamic programming is sometimes essential

Comparison with divide-and-conquer

- Divide-and-conquer algorithms split a problem into separate subproblems, solve the subproblems, and combine the results for a solution to the original problem
  - Example: Quicksort
  - Example: Mergesort
  - Example: Binary search
- Divide-and-conquer algorithms can be thought of as top-down algorithms
- In contrast, a dynamic programming algorithm proceeds by solving small problems, remembering the results, then combining them to find the solution to larger problems
- Dynamic programming can be thought of as bottom-up

Greedy Algorithms

- Greedy algorithms focus on making the best local choice at each decision point.
- For example, a natural way to compute a shortest path from x to y might be to walk out of x, repeatedly following the cheapest edge until we get to y. WRONG!
- In the absence of a correctness proof greedy algorithms are very likely to fail.

Optimization problems

- An optimization problem is one in which you want to find, not just *a* solution, but the *best* solution
- A "greedy algorithm" sometimes works well for optimization problems
- A greedy algorithm works in phases. At each phase:
  - You take the best you can get right now, without regard for future consequences
  - You hope that by choosing a *local* optimum at each step, you will end up at a *global* optimum

- Dynamic programming is a very powerful, general tool for solving optimization problems.
- Once understood it is relatively easy to apply, but many people have trouble understanding it.

Backtracking

- Suppose you have to make a series of *decisions*, among various *choices*, where
  - You don't have enough information to know what to choose
  - Each decision leads to a new set of choices
  - Some sequence of choices (possibly more than one) may be a solution to your problem
- Backtracking is a methodical way of trying out various sequences of decisions, until you find one that "works"